Rajalakshmi Engineering College

Name: M.S Nanthana shree
Email: 241901064@rajalakshmi.edu.in
Roll no: 241901064
Phone: 9360753427
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results

2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

A company is developing a user registration system that requires users to provide valid email addresses. The development team is implementing an EmailValidator program to ensure that the entered email addresses meet certain criteria using exception handling.

The email address must contain the "@" symbol.The email address must consist of a non-empty username(before "@" symbol) and a non-empty domain(after "'@" symbol).The domain part of the email address must contain at least one period (".").The email address must not contain leading or trailing spaces.

Implement a custom exception, InvalidEmailException, to fulfill the company's requirements and validate it according to the specified rules.

Input Format

The input consists of a string value 's', which represents the email address.

Output Format

The output is displayed in the following format:

If the entered email address is valid according to the specified rules, the program prints:

"Email address is valid!"

If the entered email address misses the username or domain part or misses "@" symbol or has two or more "@" symbols or misses '.' in the domain part it outputs:

"Error: Invalid email format."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: johndoe@example.com

Output: Email address is valid!

Answer

```
import java.util.Scanner;

class InvalidEmailException extends Exception {
    public InvalidEmailException() {
        super("Error: Invalid email format.");
    }
}

public class Main {

    public static void validateEmail(String email) throws InvalidEmailException {
        if (email == null || email.trim().isEmpty()) {
            throw new InvalidEmailException();
        }
```

```
        email = email.trim();
        int atCount = 0;
        for (char ch : email.toCharArray()) {
            if (ch == '@') atCount++;
        }
        if (atCount != 1) {
            throw new InvalidEmailException();
        }
        String[] parts = email.split("@");
        if (parts.length != 2 || parts[0].isEmpty() || parts[1].isEmpty() || !
parts[1].contains(".")) {
            throw new InvalidEmailException();
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String email = sc.nextLine();
        try {
            validateEmail(email);
            System.out.println("Email address is valid!");
        } catch (InvalidEmailException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Tim was tasked with creating a user profile system that validates the user's date of birth input. The system should throw a custom exception, InvalidDateOfBirthException, if the date is not in the specified format "dd-mm-yyyy" or if it represents an invalid calendar date.

The main method takes user input, validates the date of birth, and prints whether it is valid or not.

Input Format

The input consists of a string, representing the date of birth of the user.

Output Format

The output displays one of the following results:

If the entered date of birth is valid according to the specified format, the program prints:

"[Date] is a valid date of birth"

If the entered date of birth is not valid according to the specified format, the program prints:

"Invalid date: [Date]"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 01-01-2000

Output: 01-01-2000 is a valid date of birth

Answer

```
import java.util.*;
import java.text.*;

class InvalidDateOfBirthException extends Exception {
    public InvalidDateOfBirthException(String message) {
        super(message);
    }
}

public class Main {
    public static void validateDate(String date) throws
InvalidDateOfBirthException {
        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
        sdf.setLenient(false);
        try {
```

```
            sdf.parse(date);
        } catch (ParseException e) {
            throw new InvalidDateOfBirthException("Invalid date: " + date);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String dob = sc.next();
        try {
            validateDate(dob);
            System.out.println(dob + " is a valid date of birth");
        } catch (InvalidDateOfBirthException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Hemanth is designing a banking system for XYZ Bank. The system should allow customers to perform deposit, withdrawal, and balance inquiry operations. Implement exception handling for scenarios involving invalid transaction amounts or insufficient funds.

Create two custom exception classes, InvalidAmountException and InsufficientFundsException, both extending the Exception class.Throw an InvalidAmountException with a message if the deposit amount is less than or equal to zero.Throw an InsufficientFundsException if the withdrawal amount is greater than the available balance.Deduct the withdrawal amount from the balance if the withdrawal is successful.

Assist Hemanth in designing the program.

Input Format

The first line of input consists of a double value B, representing the initial balance.

The second line consists of a double value D, representing the deposit amount.

The third line consists of a double value W, representing the withdrawal amount.

Output Format

If the withdrawal is successful, print the amount withdrawn and the current balance, rounded off to one decimal place.

If an InvalidAmountException occurs, print "Error: [D] is not valid".

If an InsufficientFundsException occurs, print "Error: Insufficient funds".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1050.1
270.2
150.3
Output: Amount Withdrawn: 150.3
Current Balance: 1170.0

Answer

```
import java.util.Scanner;

class InvalidAmountException extends Exception {
    public InvalidAmountException(double amount) {
        super("Error: " + amount + " is not valid");
    }
}

class InsufficientFundsException extends Exception {
    public InsufficientFundsException() {
        super("Error: Insufficient funds");
    }
}

class BankAccount {
    private double balance;
```

```
    public BankAccount(double balance) {
        this.balance = balance;
    }

    public void deposit(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }
        balance += amount;
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException();
        }
        balance -= amount;
        System.out.printf("Amount Withdrawn: %.1f Current Balance: %.1f%n",
amount, balance);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double initialBalance = sc.nextDouble();
        double depositAmount = sc.nextDouble();
        double withdrawAmount = sc.nextDouble();

        BankAccount account = new BankAccount(initialBalance);

        try {
            account.deposit(depositAmount);
            account.withdraw(withdrawAmount);
        } catch (InvalidAmountException | InsufficientFundsException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

In an online shopping cart system, users can apply coupon codes during checkout to avail of discounts. However, to ensure the validity and security of coupon codes, the system enforces specific rules for their format. Your task is to implement a Java program named CouponCodeValidator that takes user input for a coupon code and validates it according to the specified rules.

Rules for Valid Coupon Code:

The coupon code must consist of exactly 10 characters.The coupon code must contain at least one alphabet (uppercase or lowercase) and at least one digit (0-9).Special characters are not allowed in the coupon code.

Implement a custom exception, InvalidCouponException, to handle cases where the entered coupon code does not meet the specified criteria.

Input Format

The input consists of a string s, representing the coupon code.

Output Format

The output is displayed in the following format:

If the entered coupon code meets the specified criteria, the program outputs

"Coupon code applied successfully!"

If the entered coupon code has less than or more than 10 characters it outputs

"Error: Invalid coupon code length. It must be exactly 10 characters."

If the entered coupon code contains only numeric or only alphabets it outputs

"Error: Invalid coupon code format. It must contain at least one alphabet and one digit."

If the entered coupon code contains special characters it outputs

"Error: Coupon code should not contain special characters."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ABCD123456

Output: Coupon code applied successfully!

Answer

```
import java.util.*;

class InvalidCouponException extends Exception {
   public InvalidCouponException(String message) {
      super(message);
   }
}

class CouponCodeValidator {
   public static void validateCoupon(String code) throws InvalidCouponException
{
      if (code.length() != 10)
         throw new InvalidCouponException("Error: Invalid coupon code length. It
must be exactly 10 characters.");

      boolean hasAlpha = false, hasDigit = false;
      for (char c : code.toCharArray()) {
         if (Character.isLetter(c))
            hasAlpha = true;
         else if (Character.isDigit(c))
            hasDigit = true;
         else
            throw new InvalidCouponException("Error: Coupon code should not
contain special characters.");
      }

      if (!hasAlpha || !hasDigit)
         throw new InvalidCouponException("Error: Invalid coupon code format. It
must contain at least one alphabet and one digit.");
   }
}
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String code = sc.next();
        try {
            CouponCodeValidator.validateCoupon(code);
            System.out.println("Coupon code applied successfully!");
        } catch (InvalidCouponException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Status : Correct ***Marks : 10/10***